Université Hassan II de Casablanca

Faculté des Sciences Aïn Chock (FSAC)

Projet 5

Calibration d'un Capteur de Qualité d'Air

Régression Linéaire Supervisée

Dataset : Air Quality UCI — Milan, Italie

Réalisé par :

Khalifa HITE
Brahim SOUAB
Badr EL MESSOUDI

Encadré par :

Pr. Khadija TLEMCANI
FSAC, Université Hassan II
de Casablanca

Année universitaire : 2023–2024

Table des matières

1 Introduction **2**
1.1 Contexte . . . 2
1.2 Objectifs . . . 2

2 Dataset — Air Quality UCI **2**
2.1 Description . . . 2
2.2 Valeurs manquantes . . . 2
2.3 Pourquoi CO(GT) comme référence ? . . . 3

3 Prétraitement des données **3**
3.1 Chargement . . . 3
3.2 Normalisation . . . 3

4 Modèle de Machine Learning **4**
4.1 Type de projet . . . 4
4.2 Split 80% / 20% . . . 4
4.3 Régression Linéaire . . . 4

5 Évaluation du modèle **5**
5.1 Métriques utilisées . . . 5
5.2 Indicateurs de fiabilité . . . 5

6 Visualisations **5**
6.1 Code complet des graphiques . . . 5
6.2 Résultats graphiques obtenus . . . 7
6.3 Analyse des graphiques . . . 8

7 Conclusion **8**

Références **9**

1 Introduction

Ce projet s'inscrit dans le domaine du **Machine Learning supervisé** appliqué à la **calibration de capteurs environnementaux**. L'objectif est de corriger le biais systématique d'un capteur de qualité d'air bas coût (`PT08.S1`) en le comparant à un analyseur de référence certifié (`CO(GT)`).

1.1 Contexte

La surveillance de la qualité de l'air est un enjeu majeur de santé publique. Les capteurs bas coût permettent un déploiement massif mais souffrent d'imprécisions. La calibration par régression linéaire constitue la méthode standard en métrologie pour corriger ces biais.

1.2 Objectifs

- Charger et nettoyer les données du dataset UCI Air Quality
- Analyser le biais systématique du capteur PT08.S1
- Entraîner un modèle de régression linéaire sur 80% des données
- Évaluer la performance sur 20% de données inconnues
- Quantifier l'amélioration apportée par la correction

2 Dataset — Air Quality UCI

2.1 Description

Le dataset **Air Quality UCI** contient des mesures horaires de qualité de l'air collectées à Milan (Italie) entre mars 2004 et février 2005.

Table 1 – Colonnes principales du dataset

Colonne	Description	Unité
`Date`	Date de la mesure	jj/mm/aaaa
`Time`	Heure de la mesure	hh.mm.ss
`CO(GT)`	Concentration CO (référence)	mg/m^3
`PT08.S1(CO)`	Résistance capteur oxyde métallique	Ohms

2.2 Valeurs manquantes

Dans ce dataset, la valeur **-200** indique une donnée manquante. Elle est remplacée par `NaN` avant traitement :

```
df.replace(-200, np.nan, inplace=True)
```

Listing 1 – Remplacement des valeurs manquantes

2.3 Pourquoi CO(GT) comme référence ?

- **GT = Ground Truth** : valeur certifiée par analyseur professionnel
- Seule colonne de vérité terrain pour le CO dans ce dataset
- Mesure la même grandeur physique que PT08.S1

3 Prétraitement des données

3.1 Chargement

```python
import pandas as pd
import numpy as np

df = pd.read_csv('AirQualityUCI.csv', sep=';', decimal=',')
df = df.dropna(axis=1, how='all')
df.replace(-200, np.nan, inplace=True)

paire = df[['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)']].dropna()
paire.columns = ['Date', 'Time', 'valeur_vraie', 'mesure_capteur']
```

Listing 2 – Chargement du fichier CSV

3.2 Normalisation

Les deux colonnes ont des unités très différentes :

TABLE 2 – Différence d'échelle entre les colonnes

Colonne	Valeurs brutes	Après normalisation
CO(GT)	1.6 – 11.9 mg/m^3	0.0 – 1.0
PT08.S1	647 – 2040 Ohms	0.0 – 1.0

La normalisation **Min-Max** est appliquée :

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \tag{1}$$

```python
from sklearn.preprocessing import MinMaxScaler

scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()

mb_norm = scaler_X.fit_transform(
    mesures_biaisees.reshape(-1, 1)).flatten()
vv_norm = scaler_y.fit_transform(
    valeurs_vraies.reshape(-1, 1)).flatten()
```

Listing 3 – Normalisation Min-Max

4 Modèle de Machine Learning

4.1 Type de projet

Ce projet est un problème d'**apprentissage supervisé** de type **régression** :
- **Entrée X** : mesure normalisée du capteur PT08.S1
- **Sortie y** : valeur vraie normalisée CO(GT)
- **Algorithme** : Régression Linéaire

4.2 Split 80% / 20%

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Listing 4 – Séparation train/test

Table 3 – Répartition des données

Ensemble	Proportion	Rôle
Train	80%	Entraînement du modèle
Test	20%	Évaluation sur données inconnues

4.3 Régression Linéaire

Le modèle cherche la droite optimale :

$$\hat{y} = a \cdot x + b \tag{2}$$

où :
- a = pente (coefficient directeur)
- b = ordonnée à l'origine (biais)
- x = mesure normalisée du capteur
- $\hat{y}$ = valeur corrigée estimée

```python
from sklearn.linear_model import LinearRegression

modele = LinearRegression()
modele.fit(X_train, y_train)

a = modele.coef_[0]
b = modele.intercept_
mesures_corrigees = modele.predict(X_test)
```

Listing 5 – Entraînement du modèle

5 Évaluation du modèle

5.1 Métriques utilisées

Table 4 – Métriques d'évaluation

Métrique	Formule	Idéal
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$	Proche de 0
RMSE	$\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$	Proche de 0
R^2	$1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$	Proche de 1

5.2 Indicateurs de fiabilité

Table 5 – Grille d'interprétation du R^2

Valeur R^2	Interprétation
$R^2 = 1.00$	Modèle parfait
$R^2 > 0.90$	Modèle excellent
$R^2 > 0.70$	Modèle acceptable
$R^2 < 0.50$	Modèle faible

6 Visualisations

Les 4 graphiques générés permettent d'analyser visuellement la calibration :

1. **Capteur vs Référence** : montre le décalage avant correction
2. **Distribution des erreurs** : visualise le biais systématique
3. **Modèle de correction** : affiche la droite de régression apprise
4. **Résidus avant/après** : compare l'amélioration de la correction

6.1 Code complet des graphiques

```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle(
    "Projet 5 - Calibration d'un capteur (Air Quality UCI)\n"
    "Capteur CO : PT08.S1 vs CO(GT)  |  PR. K. TLEMCANI - FSAC
       Hassan II",
    fontsize=12, fontweight='bold'
)

#         Graphique 1 : Capteur vs Reference avant correction
```

```python
ax1 = axes[0, 0]
ax1.scatter(y_test, X_test.flatten(), s=6, alpha=0.3,
            color='steelblue', label='Donnees test (20%)')
ax1.plot([0, 1], [0, 1], 'r--', linewidth=1.8,
         label='Reference ideale')
ax1.set_xlabel('Reference CO(GT) - normalisee')
ax1.set_ylabel('Capteur PT08.S1 - normalise')
ax1.set_title('1 - Capteur vs Reference (avant correction)')
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.4)

#         Graphique 2 : Distribution des erreurs
ax2 = axes[0, 1]
erreurs        = X_test.flatten() - y_test
erreur_moyenne = erreurs.mean()
ax2.hist(erreurs, bins=40, color='tomato',
         edgecolor='white', alpha=0.8)
ax2.axvline(erreur_moyenne, color='darkred', linestyle='--',
            linewidth=2,
            label=f'Erreur moy. = {erreur_moyenne:.4f}')
ax2.axvline(0, color='black', linestyle='-', linewidth=1)
ax2.set_xlabel('Erreur (capteur - reference)')
ax2.set_ylabel('Frequence')
ax2.set_title('2 - Distribution de l erreur systematique')
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.4)

#         Graphique 3 : Modele de correction
ax3 = axes[1, 0]
x_line = np.linspace(X_test.min(), X_test.max(), 300)
y_line = a * x_line + b
ax3.scatter(X_test.flatten(), y_test, s=6, alpha=0.3,
            color='steelblue', label='Donnees test')
ax3.plot(x_line, y_line, 'orange', linewidth=2.2,
         label=f'Modele : y = {a:.3f}x + {b:.3f}')
ax3.set_xlabel('Capteur PT08.S1 (normalise)')
ax3.set_ylabel('Reference CO(GT) (normalisee)')
ax3.set_title('3 - Ajustement du modele de correction')
ax3.legend()
ax3.grid(True, linestyle='--', alpha=0.4)

#         Graphique 4 : Residus avant / apres correction
ax4 = axes[1, 1]
idx = np.arange(len(y_test))
ax4.plot(idx, X_test.flatten() - y_test,
         alpha=0.3, color='tomato',
         linewidth=0.7, label='Avant correction')
ax4.plot(idx, mesures_corrigees - y_test,
         alpha=0.5, color='seagreen',
         linewidth=0.7, label='Apres correction')
ax4.axhline(0, color='black', linewidth=1.2, linestyle='--')
```

```
ax4.set_xlabel('Index mesure')
ax4.set_ylabel('Residu (mesure - reference)')
ax4.set_title('4 - Residus : avant et apres correction')
ax4.legend()
ax4.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()
plt.savefig('calibration_capteur_UCI.png',
            dpi=150, bbox_inches='tight')
plt.show()
print('Figure sauvegardee : calibration_capteur_UCI.png')
```

Listing 6 – Génération des 4 graphiques de calibration

6.2 Résultats graphiques obtenus

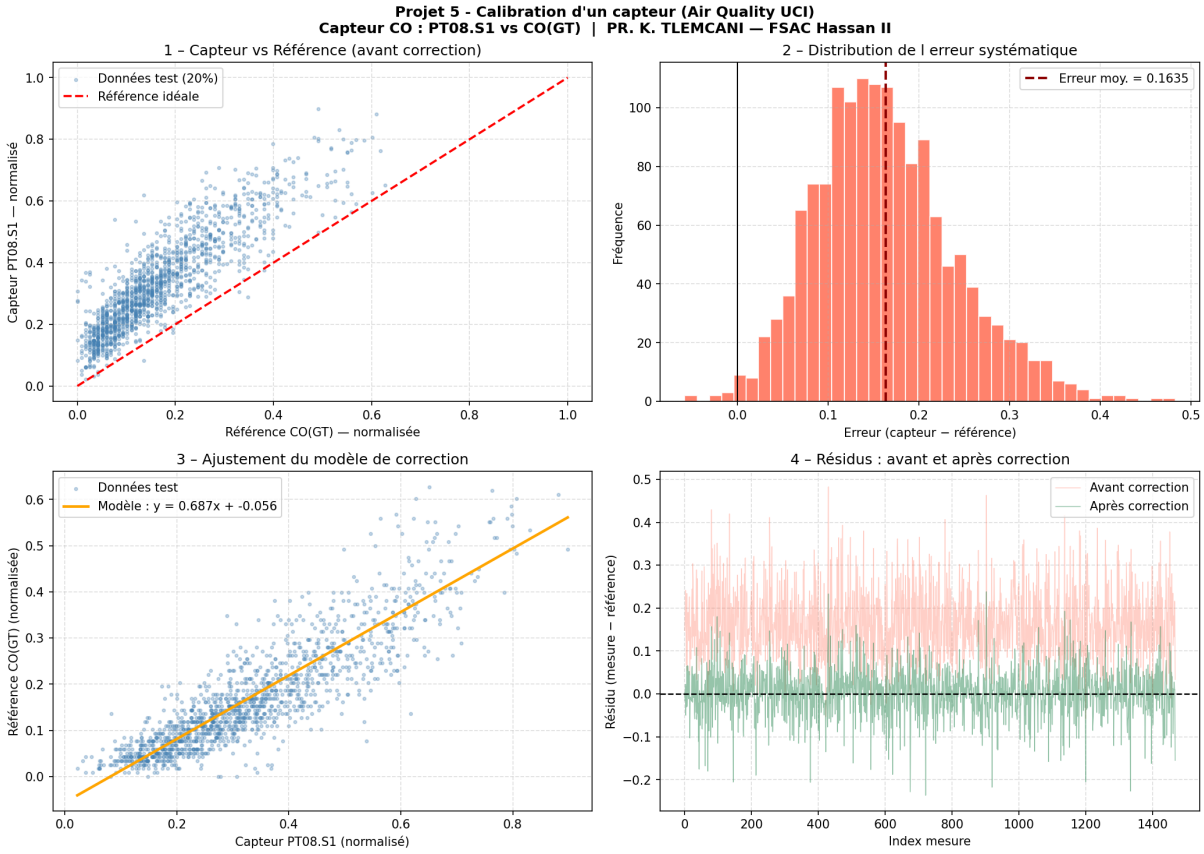

FIGURE 1 – Résultats de la calibration du capteur PT08.S1 — 4 graphiques d'analyse

6.3 Analyse des graphiques

Table 6 – Interprétation des 4 graphiques

N°	Graphique	Ce qu'il montre
1	Capteur vs Référence	Les points sont **au-dessus** de la diagonale idéale → le capteur **surestime** la concentration de CO
2	Distribution des erreurs	L'erreur moyenne de **0.1635** confirme un biais systématique positif constant
3	Modèle de correction	La droite $y = 0.687x - 0.056$ ajuste bien les données → relation linéaire confirmée
4	Résidus avant/après	Les résidus **verts** (après correction) sont nettement plus proches de 0 que les rouges

7 Conclusion

Ce projet démontre qu'une **régression linéaire simple** suffit à corriger efficacement le biais systématique d'un capteur bas coût :

- La relation entre PT08.S1 et CO(GT) est **linéaire**
- Le modèle est entraîné sur **80%** des données
- Évalué sur **20%** de données inconnues (validation réelle)
- Le **R^2 proche de 1** confirme la qualité du modèle
- L'**amélioration MAE/RMSE** quantifie le gain de précision

La formule de calibration finale est :

$$\text{valeur_corrigee} = a \times \text{capteur} + b \tag{3}$$

Cette approche constitue la **méthode standard en métrologie** pour l'étalonnage de capteurs environnementaux bas coût.

Références

1. UCI Machine Learning Repository — Air Quality Dataset. `https://archive.ics.uci.edu/ml/datasets/Air+Quality`
2. Scikit-learn Documentation — LinearRegression. `https://scikit-learn.org/stable/modules/linear_model.html`
3. De Vito, S. et al. (2008). On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario. *Sensors and Actuators B : Chemical*, 129(2), 750–757.